PRACTICAL EXERCISE 25

Using AI to construct database schemas, insert sample records, and formulate analytical queries in MySQL.

USER PROMPT

"Create a MySQL database table named students with student_id, name, course, and marks. Insert five sample records, then write queries to display all students, find the highest marks, calculate average marks, and identify students scoring above 80."

PHASE 1: DATABASE SETUP & DATA INSERTION

```
-- 1. Create the 'students' table CREATE TABLE students ( student_id INT PRIMARY KEY AUTO_INCREMENT, name VARCHAR(100) NOT NULL, course VARCHAR(100), marks DECIMAL(5,2) ); -- 2. Insert five sample records INSERT INTO students (name, course, marks) VALUES ('Alice Walker', 'Computer Science', 88.50), ('Brian Chen', 'Mathematics', 76.00), ('Chloe Smith', 'Physics', 92.00), ('David Kim', 'Computer Science', 81.50), ('Emma Jones', 'Economics', 65.00);
```

PHASE 2: ANALYTICAL QUERIES

```
-- Display all students SELECT * FROM students; -- Find the highest marks SELECT MAX(marks) AS highest_marks FROM students; -- Calculate average marks SELECT AVG(marks) AS average_marks FROM students; -- Identify students scoring above 80 SELECT name, course, marks FROM students WHERE marks > 80;
```

PHASE 3: VALIDATION & EXECUTION OUTPUT

Follow-up Task: Run each query in a MySQL environment to verify the syntax and mathematical logic generated by the AI. The simulated output below confirms the queries execute correctly.

92.00	80.60
HIGHEST MARKS (MAX)	AVERAGE MARKS (AVG)

Query Result: Students Scoring Above 80

name	course	marks
Alice Walker	Computer Science	88.50
Chloe Smith	Physics	92.00
David Kim	Computer Science	81.50

Key Learning Outcomes

1. DDL & DML Generation: AI is highly proficient at generating Data Definition Language (table creation) and Data Manipulation Language (insert statements) syntax, saving time on database scaffolding.

2. Aggregate Functions: The AI correctly identified and applied standard SQL aggregate functions (`MAX()`, `AVG()`) with proper aliasing (`AS`) to meet the analytical requirements.

3. Query Validation: Testing the AI output in a live SQL environment is critical. While AI accurately handles basic `WHERE` clauses, complex relational joins (not covered here) often require manual schema review to ensure accuracy.